CODERS IN PUNE · KNOWLEDGE HANDOVER CHALLENGE · ROUND 1: THE PLAN

Baton

Hand over the work, **not just the files.** Every claim has a receipt. Every gap gets asked. Every reader sees only what they're allowed to see.

Evidence Ledger

The handover is built from small claims, and each one links to the exact sentence it came from. A second model checks every claim against its source before it can be marked *Verified*.

Gap Interview

Baton finds what the sources *can't* explain and only the leaver knows. It then asks them a short, ranked list of questions before their last day.

Sealed Claims

Each reader's handover is built from only the sources that reader can open. Restricted items show up as a locked placeholder, so the reader knows they exist without seeing what they say.

Team **leisure** · Solo entry by **Sidharth Monangi**

Submitted **November 2026**

AT A GLANCE

Executive summary

When someone leaves a project, the documents usually stay behind. What leaves with them is the reasoning: why a decision was made, what is half-finished, who to call, and which problem keeps coming back. Baton rebuilds that reasoning from the project's authorised sources. It shows exactly where each statement came from, says clearly what it doesn't know, and turns the result into a continuation plan that a responsible person reviews and approves before the newcomer sees it.

The insight

The usual answer is an AI summary. A summary can only restate what's already written down, and it sounds just as sure of itself when it's guessing. The knowledge that's actually at risk is the part that **was never written down**. So Baton is built around finding gaps first: it works out what the sources can't support, and spends the leaver's last days filling those gaps.

The product

There are seven steps: **Connect → Reconstruct → Verify → Ask → Review → Handover → Continue**. The result is a handover tailored to each reader's role. It contains typed, cited claims, a timeline of decisions, a map of who owns what, a list of risks and dependencies, and a Day 1 / Week 1 / Month 1 plan that the incoming person signs off on.

Why it's feasible

The MVP needs just two kinds of source: the GitHub API, plus uploaded files (Slack/Discord exports, documents, meeting transcripts, task-board CSVs). For the demo we'll use a synthetic project with a known answer key, so we can measure accuracy and leakage instead of just claiming them.

100%

of claims in a shared handover cite at least one source, or are clearly labelled *Unsupported / Gap*. Nothing is presented as fact without a source.

0

confidential items leaked to unauthorised roles. Every build runs an automated leakage test that must pass.

≥ 90%

target precision on our benchmark: a claim only counts if its cited source actually supports it, as judged against ground truth.

−50%

target cut in the time a newcomer needs to answer 10 "pick up the project" questions correctly, measured in a usability test.

How this document maps to the submission checklist

1. Understanding of the problem: §1
2. Users affected: §2
3. Proposed solution: §3
4. Complete product workflow: §4 (with worked example)
5. Main features: §5
6. Technology stack and architecture: §6
7. Responsibilities (solo plan): §7
8. Development plan across all three rounds: §8
9. Expected impact: §9
10. Future scope: §10
11. Team and participant details: §11

\+ Research plan, references, AI-use disclosure: Appendix

How we line up with the judging criteria

Understanding (25): we pin down what is lost and why documentation fails (§1). **Solution (25):** each workflow step answers a specific failure (§3–4). **Technical (20):** permissions are enforced at three separate layers, and every claim is tied to an exact span of its source (§6). **Originality (15):** Gap Interview, Sealed Claims and a Sole-Knower Index, which together go well beyond summarising (§3). **Feasibility (15):** this is a solo entry, so the scope is a deliberately narrow MVP on managed services that we can demo and measure, and anything beyond it is held for Round 3 (§5, §8).

§1 · UNDERSTANDING OF THE PROBLEM

Projects don't lose documents. They lose the reasoning.

A departing member usually leaves behind a repository, a drive folder and a long chat history. What the incoming person actually needs is spread across all of them and rarely written down: why things are the way they are, what's half-done, and what will go wrong next.

42%

of institutional knowledge is unique to the individual employee. It was learned doing the role and isn't shared with co-workers.
Panopto, Workplace Knowledge & Productivity Report (survey of 1,000+ US workers, 2018) [R1]

5.3 h

per week is how long knowledge workers spend waiting for information from colleagues, or recreating information that already exists.
Panopto, same study [R1]

0.5–2×

a person's annual salary is what it costs to replace one employee, and Gallup calls that a conservative estimate. Lost knowledge is a large hidden part of that cost.
Gallup, "This Fixable Problem Costs U.S. Businesses $1 Trillion" [R2]

What exactly is lost

Lost knowledge	Example (student tech-club app)	Where it hides today
Decision rationale	"We dropped Firebase because the free tier broke during the fest registration spike."	A 2 a.m. Slack thread, never put in the README
Unfinished work	Half-merged payment PR; a migration that ran on staging only	Open PRs, local branches, the leaver's memory
Ownership & contacts	"Ask Meera at the sponsor for the API quota; she replies only on email."	The leaver's personal relationships
Dependencies	The college SSO certificate expires in March and breaks login	One calendar reminder on one person's phone
Recurring problems	Deploys fail on Fridays because the free CI runner hits its quota	Repeated across 6 issues that nobody has linked together
Reversed decisions	The design doc says "MongoDB"; the code has used Postgres since April	Contradicting sources, with no record of which is current

Why existing documentation and tools fail

- **Docs capture *what*, rarely *why*.** Reasoning happens in conversation, and conversation is the part nobody curates.
- **Handover notes get written at the worst moment:** in the last week, by someone busy and already checked out. They suffer from the *curse of knowledge*, which means the writer can't see what a newcomer doesn't know.
- **Docs go stale quietly.** A wiki page written in January says nothing about the April reversal, and doesn't warn you that it's out of date.
- **Generic AI summarisers sound equally sure when they're right and when they're guessing.** They tidy up contradictions instead of flagging them, and they can only summarise what was written. They never ask what's missing.
- **Search tools answer the questions you know to ask.** A newcomer's biggest risks are the questions they don't know to ask yet.
- **Permissions often stop at retrieval.** A summary can mix restricted facts (budget, HR, credentials) into text that goes to everyone.

The part of the problem we will solve

We focus on **planned and sudden departures from small and mid-sized project teams** whose work lives in a code repository plus chat, documents, meeting notes and a task board. These are student tech clubs, hackathon and open-source teams, research groups and early-stage software teams. Our goal is to **shorten the time it takes the incoming person to work safely and independently**, without ever showing them anything they aren't authorised to see. We are deliberately *not* building a general enterprise search engine, or a tool that judges the leaver's performance.

§2 · USERS AFFECTED

Four people sit around every handover

Our first market is **college technical clubs and student project teams**. Their leadership turns over almost completely every year at graduation, they run on free tools (GitHub, Slack/Discord, Google Docs, Trello/Notion), and they have no knowledge-management staff at all. The same workflow carries over without changes to volunteer NGOs, research labs and small software teams.

Departing member

Priya, outgoing tech lead, graduating in 3 weeks

Pain: she has a placement to start and exams to sit. Writing a 20-page handover feels pointless, and she doesn't know what her successor will need.
Baton gives her: a draft she only has to *correct*, plus a **20-minute Gap Interview** in place of a blank page.

Incoming member

Arjun, second-year, new tech lead

Pain: he's inheriting 400 commits, 9 channels and 30 open issues, and he's afraid to touch production.
Baton gives him: a cited handover built for his role, a Day 1 / Week 1 / Month 1 plan, a list of people to talk to, and a clear list of what's still unknown.

Reviewer / approver

Dr. Kulkarni, faculty coordinator (or an engineering manager)

Pain: the club is accountable for sponsor money and student data, and nobody can see what knowledge is at risk.
Baton gives them: a review screen where they accept, edit or reject each claim, a **Handover Readiness Score**, and an audit trail.

Workspace admin

Club secretary or IT owner

Pain: they want to use AI, but they're worried about leaking secrets, personal data or finance details.
Baton gives them: scoped source connections, sensitivity labels, role-based access policies, automatic redaction and a record of every access.

Departure and onboarding scenarios we design for

Scenario	What is different	How Baton adapts
Planned exit graduation, resignation	The leaver is available for 1–4 weeks, but busy	Full workflow, with a time-boxed Gap Interview ranked by risk
Sudden exit illness, emergency, drop-out	There's no interview, so everything rests on the sources	Gaps are routed to the next-best person ("who else touched this?"), and the uncertainty labels become more prominent
Rotation / split	One leaver hands over to several people	Each recipient gets their own handover, with ownership split by area
Temporary leave	The person is coming back	A time-bounded handover, plus a "what changed while you were away" summary on return
Mixed audience	Successor, junior volunteers and the sponsor contact all need different views	Each view is built for its reader's role, and restricted items appear as Sealed Claims

Environments

Baton is a browser-based web app with no installation, so it runs on any laptop. Sources are connected through OAuth (the GitHub App, which only reads) or by uploading exports, which suits teams that can't grant live access. Hosting is in the cloud for the hackathon. Because the architecture is containerised, a self-hosted deployment for institutions with strict data rules is a realistic next step (§10).

§3 · PROPOSED SOLUTION

Baton: a handover built from evidence, gaps and permissions

Baton doesn't write one long summary. It builds an **Evidence Ledger**, which is a structured set of small claims, each tied to exact source spans. Every handover is put together from that ledger, and its gaps are filled by the one person who can fill them.

1 Evidence Ledger with trust states

An extraction model reads each source and pulls out small claims of 8 types: *Decision, Ownership, Open Question, Risk, Dependency, Blocker / Recurring Problem, Next Action, Contact*. Each claim points to exact character spans in its sources. A **separate verifier model** then checks whether those spans actually support the claim. Every claim ends up with exactly one visible trust state:

VERIFIED · INFERRED · CONFLICTING · UNSUPPORTED / GAP · ATTESTED · SEALED

An assumption can never look like a fact. Only *Verified* and *Attested* (confirmed by a named person, with a date) claims are shown as plain statements. Everything else is labelled where it appears.

2 Gap Interview (our key idea)

Baton looks for **what the evidence can't explain**: decisions with no recorded reason, tasks with no owner, contradictions, risks with no mitigation, and areas that **only the leaver ever touched**. It ranks these by risk and turns the top 10–15 into short, specific questions for the leaver, for example: *"PR #142 switched auth to college SSO. Why, and what still has to be done before the March certificate renewal?"*

Answers come in as typed text (voice is planned for Round 3) and are stored as ATTESTED evidence with the person's name and the date. Twenty minutes answering targeted questions replaces days of writing on a blank page.

3 Sealed Claims: permissions applied when each handover is built

Every span of evidence keeps the access list of the source it came from. When we build a handover for a particular reader, a claim is included only if **(a)** that reader can open at least one of its sources, **(b)** the claim still passes verification using *only* those sources, and **(c)** its sensitivity level is within the reader's clearance. If a claim fails these checks but matters, the reader sees a locked placeholder in its place: 🔒 **1 finance item — request access from Treasurer**. The reader knows something is there, and none of its content leaks.

4 Sole-Knower Index and Readiness Score

For each area (a module, a vendor, a process), Baton measures how much of the activity came from the leaver alone. This works like a per-topic "bus factor": how many people would have to leave before nobody understands that area. Combined with how many critical gaps are still open, it gives a **Handover Readiness Score** (0–100) that the reviewer can watch go up as gaps are answered and claims are approved. This turns "is the handover good enough?" into a number.

How Baton compares

Capability	Wiki / handover template	Generic AI summariser	Enterprise AI search	Baton
Captures *why* decisions were made	If someone writes it	If it's in the text	If it's in the text	Yes, plus a Gap Interview
Citation on every claim, checked by a verifier	—	Sometimes	Cites documents, not individual claims	Yes, exact span + entailment check
Flags missing, conflicting or uncertain information	—	Smooths it over	—	Trust states + a list of conflicts
Asks the leaver about what's missing	—	—	—	Questions ranked by risk
Permissions applied to the *output*, not just retrieval	By hand	—	Retrieval only	Rebuilt per reader + Sealed Claims
Human review, versions, acknowledgement	Partial	—	—	Claim-level review, version diffs, sign-off
A concrete plan for continuing the work	Free text	—	—	Day 1 / Week 1 / Month 1 with owners

These comparisons describe each category in general terms, not specific vendors. Some enterprise tools do enforce permissions when retrieving documents. Baton's difference is that it re-checks every claim against the specific reader's permitted sources when it builds the final handover.

§4 · COMPLETE PRODUCT WORKFLOW

From connected sources to an approved, acknowledged handover

The five steps the brief requires are marked with a dark bar. The two amber steps are our additions, and they are what makes Baton more than a summariser.

01
Connect
Admin starts a handover: *leaver, recipients, roles, last day*. They connect GitHub or upload exports, and a consent screen shows exactly what will be read.

02
Reconstruct
Sources are split into evidence units (with author, time, access list and link) and redacted. Claims are extracted, and Baton builds the timeline, ownership map and dependency graph.

03
Verify
The verifier checks each claim against its source spans. Baton applies trust states, finds conflicts and reversed decisions, and computes the Sole-Knower Index.

04
Ask
The Gap Interview: the leaver answers the ranked gap questions. Answers become Attested evidence, and the Readiness Score goes up.

05
Review
The leaver and the approver accept, edit or reject each claim, resolve conflicts and mark items sensitive. Then they approve a version.

06
Handover
A separate handover is built for each recipient, with Sealed Claims where needed. The share link requires sign-in, and the recipient acknowledges each critical item.

07
Continue
The recipient works through the plan. New questions get answered from the approved handover, and anything it can't answer goes to the leaver or another owner.

What happens at each step

Step	System behaviour	Human control	Trust safeguard
Connect	The GitHub App (read-only) pulls commits, PRs, reviews, issues and project boards. Uploads accepted: Slack/Discord export JSON, .md/.docx/.pdf, .vtt/.txt meeting transcripts, Trello/Jira CSV. Each source is tagged with its original access list (repo collaborators, channel members) or the access list set by the person uploading it.	The admin picks which sources and dates are in scope. Private DMs are excluded by default.	A redaction gate removes secrets and personal data (API keys, phone numbers, Aadhaar-like IDs) **before any model sees the text**.
Reconstruct	The extraction model returns claims as structured JSON that must match a schema: type, text, owner, due date, source span IDs. Duplicate claims are merged by comparing their embeddings. A claim is marked as *superseded* when a later source reverses it.	—	Any claim that doesn't point to a span is thrown out, never shown.
Verify	A separate verifier model scores whether each span supports, contradicts or is unrelated to the claim. Rules then set the trust state, the conflict list and the Sole-Knower Index for each area.	—	A claim that fails verification is downgraded to *Inferred* or *Unsupported*. It is never quietly kept as fact.
Ask	Gaps are ranked by *risk × sole-knower × urgency*. Baton writes 10–15 questions, each showing the evidence behind it, so the leaver answers quickly in context.	The leaver can skip a question, mark it "don't know", or hand it to someone else.	Answers are stored as Attested (with name and date) and are never mixed up with machine claims.
Review	A review screen lists claims grouped by type and sorted by risk. Each edit is stored as a diff. When a version is approved it is hashed and frozen.	Accept, edit, reject, change the label, merge, or re-open a claim.	Nothing can be shared until a named approver signs off. Every action is recorded in an append-only log.
Handover	A separate handover is built from the approved version for each reader (successor, junior volunteer, external contact). It includes the continuation plan, a "people to talk to" list and an explicit unknowns section.	The recipient acknowledges each critical item, or flags it as unclear.	Permissions are re-checked when the handover is built and again in the database. Restricted claims appear sealed.
Continue	"Ask Baton" answers questions using only the approved handover and the reader's permitted sources. If the evidence isn't there, it says *"not in evidence"* and names who probably knows.	Unanswered questions go back to the owner.	It never answers from general knowledge or from sources the reader can't access.

§4 · WORKED EXAMPLE (DESIGN MOCK-UP)

What Arjun sees on Day 1

The sample project is **Project Orion**, a synthetic student-club event app that we build for the demo. Priya is leaving, Arjun is taking over as tech lead, and Rohan is a first-year volunteer. Below, Arjun's handover is on the left. On the right is part of Rohan's handover, built from the same approved version but for a different role.

Orion handover · v3 approved by Dr. Kulkarni · for **Arjun (Tech Lead)** — Readiness 86

DECISION VERIFIED

The backend moved from Firebase to Supabase on 10 Apr because the free tier failed during fest registrations.

`slack#backend · 03 Apr` `PR #118` `notes/04-08.vtt 12:41`

DECISION CONFLICTING superseded

The design doc says *MongoDB*, but all migrations since April target *Postgres*. Current state: **Postgres** (newest source). The doc is out of date.

`docs/design.md §3 (Jan)` `migrations/0007 (Apr)`

RISK · DEPENDENCY ATTESTED · Priya, 2 Nov

The college SSO certificate expires on 14 Mar. Login breaks unless the IT cell renews it. Contact: Mr. Deshpande (IT), on email only.

`Gap Interview Q3` `PR #142`

RECURRING PROBLEM INFERRED · 6 sources

Friday deploys fail repeatedly, probably because the CI runner hits its quota. This pattern was spotted across issues, not stated anywhere.

`#97` `#103` `#121` `+3`

OPEN QUESTION UNSUPPORTED / GAP

Nobody owns the payment webhook retry logic. Priya marked this "don't know" and it has been routed to Kabir (last committer).

Ask Kabir · Assign owner

CONTACT 🔒 SEALED

1 item about the sponsor agreement is restricted to Treasurer + Faculty. Request access

Same version · for **Rohan (Volunteer)**

NEXT ACTION · WEEK 1 VERIFIED

Fix the event card layout bug (#130). Good first issue. Mentor: Arjun.

`#130` `board: To-do`

🔒 3 items sealed

Infrastructure & finance items are not part of your role.

Gap Interview · Priya · 4 of 12 answered — ~14 min left

Q5 · High risk · Only you touched this

"You wrote 94% of `/payments`. PR #151 is open with a failing test. Should it be merged, rewritten or dropped? Who should Arjun talk to at Razorpay support?"

Type your answer...

Skip · Don't know · Hand to someone else

The continuation plan (generated, then edited by the reviewer)

Day 1

- Get access: Supabase, Vercel, GitHub admin `#ops`
- Read 5 key decisions (≈10 min)
- Acknowledge 4 critical risks

Week 1

- Decide what happens to PR #151 (payments)
- Meet Kabir about webhook ownership
- Fix the CI quota (recurring Friday failures)

Month 1

- Email IT about the SSO certificate renewal (due 14 Mar)
- Update design.md to reflect Postgres
- Re-run Baton: did the Readiness Score hold?

§5 · MAIN FEATURES WE INTEND TO BUILD

A narrow core that runs end to end, then three planned upgrades

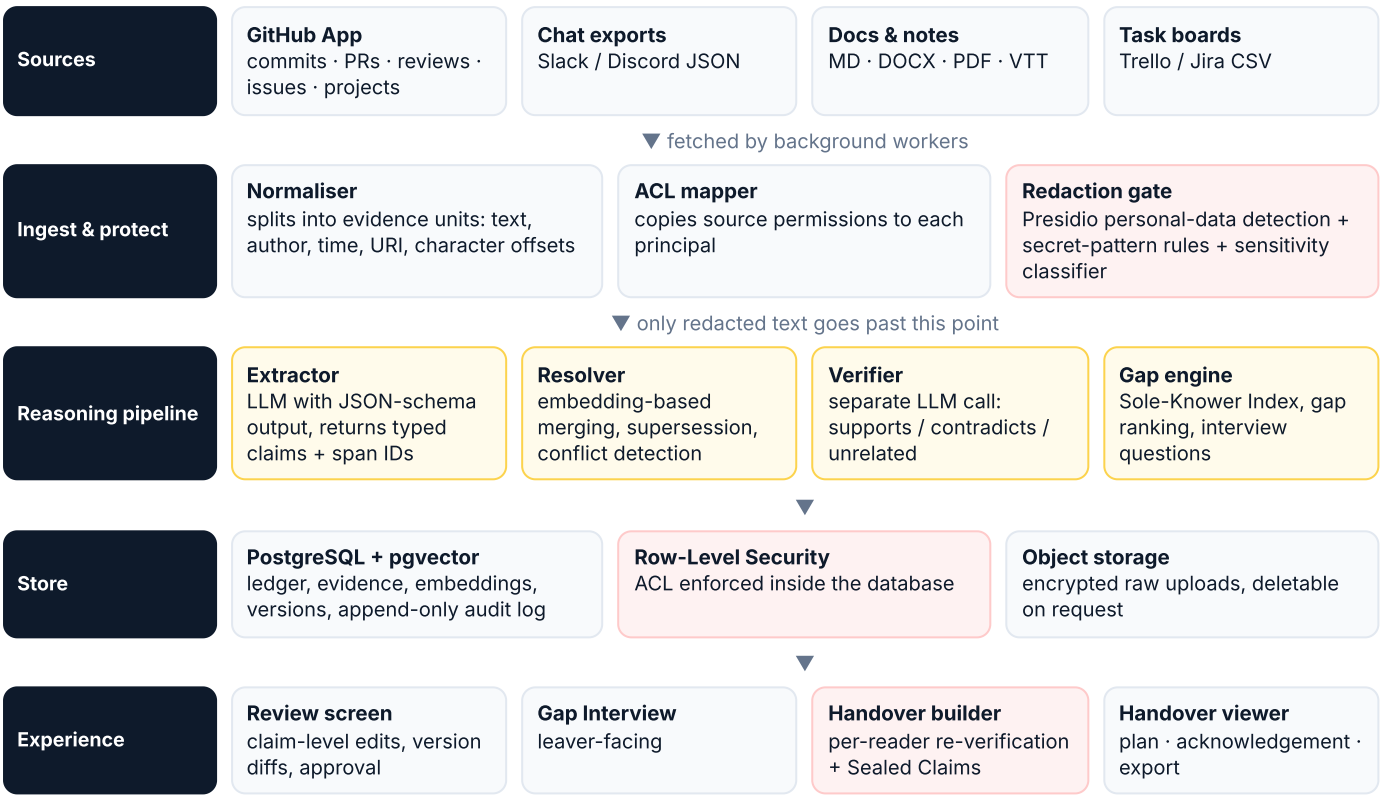
The brief says a focused workflow beats a long feature list, so we split the work into a **Round 2 MVP** that covers the whole workflow, and **three Round 3 improvements** that we commit to now.

#	Feature	What it does	Round
F1	**Handover workspace**	Create a handover with the leaver, recipients, roles, last day and scope. Includes a consent screen and an audit log.	R2 MVP
F2	**Source ingestion**	A read-only GitHub App connector (commits, PRs, reviews, issues, projects), plus uploads for Slack/Discord JSON, MD/DOCX/PDF, VTT transcripts and Trello/Jira CSV. All sources are turned into evidence units with their access lists.	R2 MVP
F3	**Redaction gate**	Detects secrets and personal data before anything reaches a model, and suggests a sensitivity label (Public / Internal / Confidential / Restricted) that a human confirms.	R2 MVP
F4	**Evidence Ledger**	Extracts the 8 claim types with exact source spans, merges duplicates, and marks claims superseded when a later source reverses them.	R2 MVP
F5	**Verifier & trust states**	An independent check that each claim is supported by its sources. Assigns the 6 trust states, a conflict list and a confidence score.	R2 MVP
F6	**Timeline, ownership map & dependency graph**	A timeline of decisions, a who-owns-what table, and a dependency and blocker graph. Every element in them links to its evidence.	R2 MVP
F7	**Gap detector + Gap Interview (text)**	Calculates the Sole-Knower Index, ranks gaps, and generates the interview questions. Answers become Attested evidence.	R2 MVP
F8	**Review screen**	Accept, edit, reject, change label or merge claims. Edits are stored as diffs, versions can be compared, and a named approver signs off.	R2 MVP
F9	**Role-based handover with Sealed Claims**	Builds a separate handover for each reader from their permitted sources, including the continuation plan and the unknowns section. Shared through an authenticated link, with acknowledgement for each critical item and export to PDF/Markdown.	R2 MVP
F10	**Readiness Score**	A live 0–100 score that combines gap coverage, how many claims have been approved and the Sole-Knower Index.	R2 MVP
R3-1	**Ask Baton (Q&A grounded in evidence)**	The incoming person asks questions after the handover. Answers come only from the approved handover and permitted sources, and include citations. When the evidence isn't there it says "not in evidence" and passes the question to the right owner.	R3 commit
R3-2	**Live sync and stale-handover alerts**	Live OAuth connections to Slack/Discord, Google Drive and Jira/Linear, with incremental syncing. Baton alerts the owner when new activity contradicts or extends an approved handover, e.g. "v3 is stale: 4 claims changed".	R3 commit
R3-3	**Voice Gap Interview + quality dashboard**	Spoken answers are transcribed and saved as Attested evidence with timestamps. A public evaluation dashboard shows precision, recall and leakage on our benchmark for every release.	R3 commit

If we need to cut scope, we'll cut in this order: F6 graph view (fall back to a table) → DOCX/PDF parsing (Markdown only) → Discord export. The **Ledger → Verify → Ask → Review → role-based handover** path will not be cut.

§6 · TECHNOLOGY STACK AND ARCHITECTURE

Architecture: every claim traceable, permissions enforced at three layers



Technology stack

Layer	Choice and why
Frontend	**Next.js + TypeScript**, Tailwind, shadcn/ui; React Flow for the graph view
API & workers	**FastAPI (Python)** + Pydantic claim schemas; Redis + RQ workers for ingestion
Database	**PostgreSQL + pgvector** (Supabase), with Row-Level Security for permissions
Auth	Supabase Auth (Google / GitHub OAuth), role claims in the JWT
LLMs	**Claude (Anthropic) API**. A larger model handles extraction and plan writing with structured output; a smaller, separate model handles verification. Calls go through a provider adapter so models can be swapped.
Embeddings	Open-source sentence-transformer (e.g. bge-small), so embedding runs locally at no cost
Privacy	Microsoft Presidio, plus regex/entropy rules to catch secrets
Parsing	GitHub REST/GraphQL, pdfplumber, python-docx, webvtt-py
Hosting & CI	Vercel (web), Render/Railway (API + worker), GitHub Actions running leakage & eval tests

Evidence model (core tables)

```
EvidenceUnit(id, source_id, uri, char_start, char_end,
  author, ts, acl[principal], sensitivity,
redacted_text)
Claim(id, type, text, owner, due, trust_state,
  confidence, origin: model|human, superseded_by)
ClaimEvidence(claim_id, evidence_id,
  relation: supports|contradicts, verifier_score)
Gap(id, area, kind, risk, sole_knower, question,
status)
Attestation(gap_id, person, answer, ts)
Review(claim_id, actor, action, before, after, ts)
Version(id, snapshot_hash, approved_by, ts)
Share(version_id, recipient, role, built_at, ack[])
AuditLog(actor, action, object, ts) — append-only
```

The visibility rule

$$\text{visible}(c, r) \Leftrightarrow E_r = \{e \in E(c) : r \in \text{acl}(e)\} \neq \emptyset \;\wedge\; \text{verify}(c, E_r) = \text{supports} \;\wedge\; \text{sensitivity}(c) \leq \text{clearance}(r)$$

otherwise → SEALED (or hidden if Restricted)

In words: reader *r* sees claim *c* only if r can open at least one of c's sources, those sources alone still support c, and c's sensitivity is within r's clearance. The citations shown are only the ones r can open. This rule is enforced in three places: the handover builder, Postgres RLS, and a CI test that fails the build if any planted secret reaches an unauthorised role.

§6 · PRIVACY, SECURITY, TRACEABILITY AND HUMAN REVIEW

Built to the brief's trust requirements

Trust requirement (from the brief)	How Baton meets it
Don't present generated assumptions as verified organisational truth	Every claim has one of six visible trust states. Only *Verified* (passed the independent check) and *Attested* (a named person, with a date) appear as plain statements. Claims without a source span are thrown out. Model-written and human-written text are always labelled separately.
Don't expose confidential information just because it appeared in a connected source	Redaction happens before any model call. Each source keeps its access list, sensitivity labels are confirmed by a person, and handovers are rebuilt per reader with Sealed Claims. RLS backs this up in the database. Private DMs are excluded by default.
Let authorised users correct, reject and approve the final handover	Claim-level accept / edit / reject / change label, with diffs, version history and a named approver. Nothing can be shared before approval. The recipient acknowledges what they've received.
Use synthetic, public, permissioned or anonymised data for demos	All demo data comes from **Project Orion**, a synthetic project we create ourselves (see §9). Live connectors are only demoed on our own test repositories.

Security

- The GitHub App asks only for read scopes, and tokens are encrypted when stored
- Uploads are stored encrypted; a "delete handover" option erases the raw data
- Share links require sign-in, expire, and can be revoked
- Append-only audit log of every view and edit

Responsible design

- The leaver gives consent and can review everything that's attributed to them
- **Baton is not a performance or surveillance tool:** it has no per-person productivity metrics
- We use API settings where providers don't train on our data
- Built with India's DPDP Act 2023 in mind (purpose limitation, deletion) [R3]

AI reliability

- Structured output is checked against a schema; malformed output is retried, then dropped
- The model that verifies claims is separate from the one that extracted them
- A fixed benchmark is re-run on every change to prompts or models
- When evidence is missing, Baton says *"not in evidence"* instead of guessing

Key technical risks and mitigations

Risk	Mitigation
The model invents claims (hallucination)	Claims must point to a source span, a separate model verifies them, the trust state is shown where the claim appears, and a human reviews before sharing
Restricted information leaks into another reader's handover	Handovers are rebuilt per reader, RLS enforces access in the database, and a CI leakage test with planted secrets fails the build
We can't get access to real organisations' data	The MVP relies on uploads and a synthetic dataset; the only live connector is GitHub on repos we own
LLM cost and latency on large projects	Claims are extracted once per evidence unit and cached; the smaller model verifies; incremental re-processing; a project the size of the demo should cost well under ₹100 per run
Scope creep during the build window	Fixed MVP, the cut-order list in §5, and the Round 3 commitments locked in advance

§7 · RESPONSIBILITIES OF EACH TEAM MEMBER

One builder, four workstreams

Team **leisure** is a solo entry: **Sidharth Monangi** owns every part of Baton, from research to deployment. The plan is scoped for one person, and each workstream below has a clear deliverable and a planned share of the build time.

Workstream	What I'm responsible for	Share of build time
Product & UX	User research and interviews, workflow design, the Review screen and Gap Interview UX, the Project Orion storyline, documentation, pitch and judge demo	20%
AI pipeline	Extraction and verifier prompts and schemas, merging duplicates and detecting conflicts, gap ranking and the Sole-Knower Index, the evaluation harness (precision, recall, leakage)	30%
Backend, integrations & security	FastAPI services, GitHub App and file parsers, data model, permission mapping, Postgres RLS, redaction gate, the per-reader handover builder, audit log, deployment	30%
Frontend & quality	Next.js app: workspace, timeline, ownership and dependency views, handover viewer and acknowledgement, accessibility (WCAG AA), end-to-end tests and the CI leakage test	20%

How a solo builder keeps this feasible

Use managed services, not infrastructure

Supabase provides the database, authentication, Row-Level Security and file storage in one place, and Vercel hosts the front end. Almost no time goes into running servers, so nearly all of it goes into the parts that make Baton different: the Ledger, the Gap Interview and Sealed Claims.

Build in vertical slices

Each sprint ends with the full workflow running end to end, even if it's rough at first. A judge can try the product at any point, and nothing important is left to integrate at the last minute.

A fixed cut order

If time runs short, features are dropped in the order listed in §5. The core path (Ledger → Verify → Ask → Review → role-based handover) is never cut.

AI-assisted development, disclosed

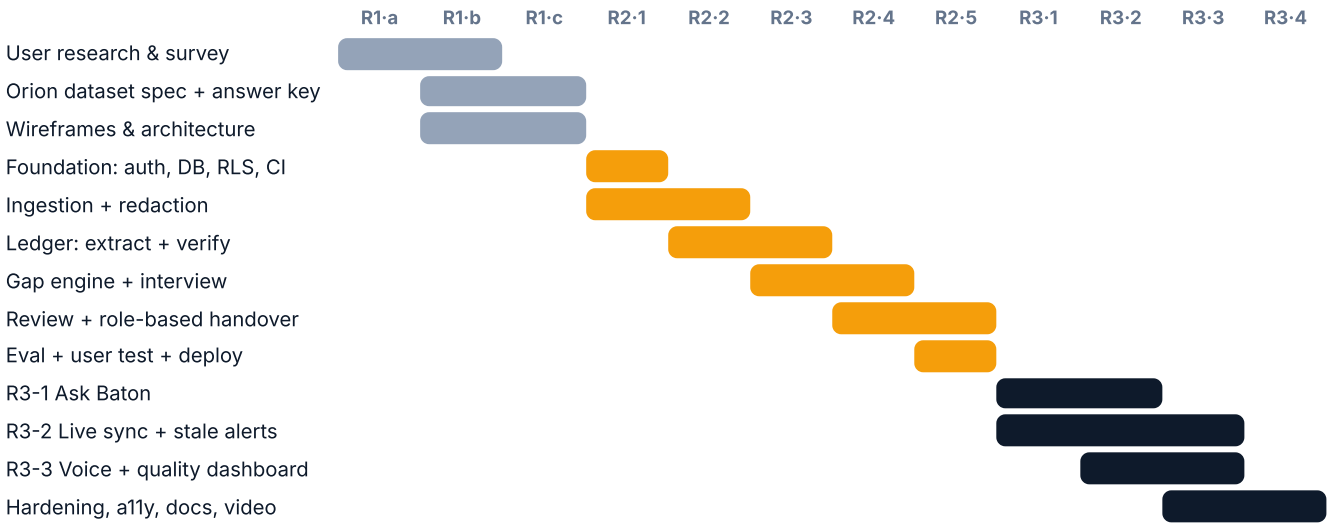
I'll use AI coding assistants to move faster, as the rules allow, and disclose them in the repository. All design decisions, testing and evaluation are mine, and the commit history will show the work being done during the build window.

Working rhythm: a GitHub Projects board, small pull requests with descriptive messages, and a short decision note for every architecture choice. My own commit, issue and decision history also gives Baton a realistic second dataset to test on, alongside Project Orion.

§8 · DEVELOPMENT PLAN ACROSS ALL THREE ROUNDS

Plan → Build the core → Finish and prove it

In line with the participation rules, **no product code is written before the official build window**. Round 1 covers research, design and specifications only. All later work will happen in a public repository with a full commit history, and any templates and AI tools we use will be disclosed.



Columns are relative phases. They will be mapped to the organiser's actual dates once those are published.

Round 1 · The Plan

Thinking, evidence, design

- Interviews and survey with club leads and successors (Appendix A)
- Project Orion specification and ground-truth answer key
- Low-fidelity wireframes for all 7 steps
- Architecture, data model, permission rule
- **Deliverable:** this document

Round 2 · The Build

The core runs end to end

- F1–F10 live on a deployed URL
- A judge can upload the Orion files or connect a repo, then get a cited, reviewable handover
- First benchmark scores published
- Usability test with 5 students
- **Deliverable:** MVP, repo, updated document, 3 commitments

Round 3 · The Finish

Complete, reliable, documented

- R3-1 Ask Baton · R3-2 Live sync · R3-3 Voice and dashboard
- Failure-case testing: empty sources, contradictions, sudden exit with no interview
- Accessibility pass, setup guide, limitations
- **Deliverable:** final product, demo video, report

Definition of done for the Round 2 demo

A judge with no walkthrough should be able to: **(1)** create a handover, **(2)** upload the Orion files, **(3)** see claims with clickable citations and trust states, **(4)** answer two Gap Interview questions as Priya, **(5)** edit and approve as the reviewer, and **(6)** open the Tech Lead and Volunteer views side by side and see the sealed items. The whole run should take under 10 minutes.

§9 · EXPECTED IMPACT

Measured, not just claimed

The final evaluation principle rewards trustworthy limitations over unsupported claims. So we'll test Baton against a benchmark we design to have a known answer key, and with real users doing timed tasks.

Project Orion benchmark (synthetic)

A synthetic 3-month history of a 6-person club project: about 150 artefacts (commits, PRs, issues, a Slack export, 6 meeting transcripts, 3 docs and a Trello CSV). We plant **~50 ground-truth facts** in it:

- 12 decisions, 3 of them later reversed
- 6 contradictions across sources
- 8 risks and 6 dependencies
- 8 sole-knower facts that exist *only* in the leaver's head (and should come up as Gap questions)
- 6 confidential items (sponsor amount, a fake API key, an HR note) that must **never** reach the Volunteer view

Metrics and targets

Metric	Target
Citation precision (cited source supports claim)	**≥ 90%**
Recall of planted facts	**≥ 75%**
Contradictions flagged	**≥ 5 of 6**
Sole-knower facts turned into Gap questions	**≥ 6 of 8**
Confidential items leaked to unauthorised roles	**0** (fails the build)
Time for a newcomer to answer 10 "take-over" questions correctly	**−50%** vs raw sources

Who benefits and how

Incoming member

Can start contributing in days rather than weeks, knows which landmines to avoid, and can see clearly what is still unknown.

Departing member

Spends about 20 minutes answering questions and correcting a draft instead of writing pages, and leaves with their work properly credited.

Organisation

Keeps sponsor relationships, credentials and context through every graduation, and can track a Readiness Score for each handover.

Ecosystem

Student clubs, NGOs and open-source projects get an option that's free and puts privacy first. These groups have the most turnover and the fewest tools today.

All targets above are hypotheses we'll test in Round 2 and report honestly in Round 3, including any targets we miss.

§10 · FUTURE SCOPE AFTER THE EVENT

From one-off handovers to continuous protection

1 · Always-ready handovers

Baton runs all the time, not only when someone resigns. Teams get an always-current handover for every key person and an organisation-wide **knowledge-risk heatmap** that shows where only one person understands something, before anyone leaves.

2 · A permission-aware knowledge layer for AI assistants

Approved handovers become available to other AI assistants through an MCP server, with the same Sealed Claims rule. Any assistant could then answer "why did we do this?" from evidence a human has approved.

3 · More sources and more settings

Microsoft Teams, Confluence, Notion, Linear, email (with consent), and Google Meet transcripts. Also new settings: research labs (when a PhD student graduates), NGOs (volunteer rotation) and open-source maintainers handing over a project.

4 · Self-hosted, privacy-first edition

A Docker deployment that runs entirely on an institution's own servers, with an option to use local open-weight models. It would be aligned with the DPDP Act, and suits colleges, government projects and regulated industries.

Sustainability model

A **free tier for student clubs, NGOs and open-source projects**, a paid team plan for startups and small and mid-sized businesses (more sources, longer history, SSO), and a self-hosted licence for institutions. The open evaluation benchmark (Project Orion) will be released publicly so other teams can measure their handover tools the same way.

Known limitations (stated up front)

- Baton can only reconstruct what was recorded somewhere, or what the leaver attests to. Knowledge from purely verbal conversations that nobody remembers can't be recovered. Baton will flag those gaps, but it can't fill them.
- Permissions copied from export files (e.g. Slack JSON) are only as accurate as the export. Admins confirm them when uploading.
- The verifier reduces errors but doesn't remove them. That's why human approval is required, not optional.

§11 · TEAM AND PARTICIPANT DETAILS

The team

Field	Details
Team name	**Ieisure**
Team ID / registration no.	To be assigned by the organisers
Team size	1 (solo entry)
Participant	**Sidharth Monangi**: owner of all workstreams (product & UX, AI pipeline, backend & security, frontend & quality)
Email	sidharthmonangi@gmail.com
GitHub	github.com/SidharthMonangi
Project repository	github.com/SidharthMonangi/Codersinpune- (judge access from Round 2)

Appendix A · Primary research plan

I'm interviewing people on both sides of real handovers in Pune college clubs and small teams, and running a short survey. Results will be reported in Round 2.

Interview guide (20 min)

- Tell us about the last time you took over (or handed over) a project.
- What did you only find out *after* the previous person had left?
- Where did the missing information actually turn out to be?
- What would you never want shared with a successor or volunteer?
- How long did it take until you felt confident making changes?

Survey (target n ≥ 40)

- Weeks to feel productive after taking over (scale)
- Hours the leaver spent on handover (scale)
- Have you ever lost access to credentials, contacts or decisions? (Y/N + example)
- Rank the most damaging losses: rationale, unfinished work, contacts, credentials, recurring issues
- Would you trust an AI handover if every claim were cited and reviewed? (Likert)

Status: interviews and the survey are under way. Findings, including the number of respondents and any results that change the plan, will be reported in the Round 2 document.

Appendix B · References

[R1] Panopto (2018). *Workplace Knowledge and Productivity Report*: 42% of institutional knowledge is unique to the individual; 5.3 hours/week spent waiting for or recreating information. panopto.com

[R2] McFeely, S. & Wigert, B. (2019). *This Fixable Problem Costs U.S. Businesses $1 Trillion*. Gallup Workplace. gallup.com/workplace/247391

[R3] Government of India (2023). *Digital Personal Data Protection Act, 2023*. meity.gov.in

Appendix C · Disclosure of AI and tools

AI assistance (Claude) was used to help structure and draft this planning document. All product decisions and research were made and reviewed by the author. In the build rounds we will use the open-source libraries and APIs listed in §6 under their licences, disclose any templates or boilerplate, and write no product code before the official build window.